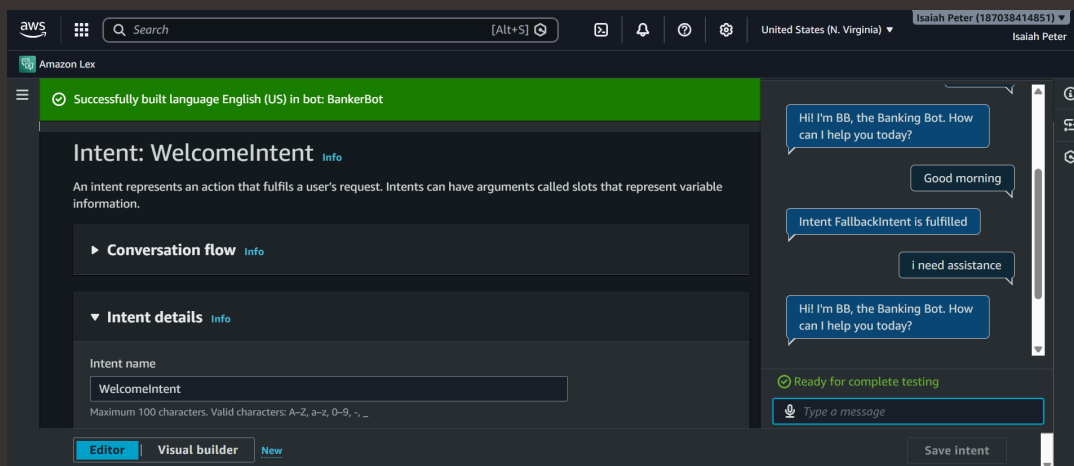




Build a Chatbot with Amazon Lex



Isaiah Peter





Introducing Today's Project!

Project overview

Today, I am starting the first project of the [NextWork Lex Chatbot series] to build my very own chatbot, **BankerBot**, using [Amazon Lex]. My goals for this project are to: * **Demonstrate my technical AI skills:** I want to showcase my practical, hands-on abilities to potential employers. * **Gain experience with AWS:** I am looking to work directly with cloud services, specifically seeing how Lex simplifies the development of conversational AI. By the end of this hour, I will have set up a chatbot capable of greeting users and handling basic interactions, which I will document for my professional portfolio!

What is Amazon Lex?

Amazon Lex is the AWS service I am using to build **BankerBot**. It is a powerful tool for designing conversational AI, allowing me to create chatbots that understand natural language and can handle both voice and text interactions. By using Amazon Lex, I'm able to define **intents** (the goals a user wants to achieve, like checking a balance or transferring funds) and manage the conversational flow without having to build complex infrastructure from scratch. It essentially provides the "brain" for my chatbot, enabling it to recognize what a user is asking and respond intelligently.

Key tools and concepts

In this project, I gained hands-on experience building a conversational AI interface using Amazon Lex.



My focus was on understanding how to translate user goals into actionable bot responses. Key takeaways included: Core Concepts: I defined Intents (user goals) and Utterances (trigger phrases) to structure the bot's intelligence. Confidence Management: I maintained a 0.40 confidence threshold, ensuring the bot only responds when it understands a request, defaulting to the FallbackIntent otherwise. User Experience: I customized the FallbackIntent with Variations to provide dynamic, helpful clarification instead of generic error messages. Infrastructure: I configured IAM roles for secure permissions, setting the stage for future integration with services like Lambda.

Personal reflections

One thing that surprised me about this project was how much the **intent classification confidence score** actually dictates the "intelligence" of the chatbot. It was eye opening to see that simply building the bot isn't enough, I had to actively manage the balance between the bot being helpful and being too "guessy." Realizing that if the confidence score is too low, the **FallbackIntent** triggers—and then learning how to use that to my advantage by adding **variations**—really highlighted how much effort goes into making a bot feel natural and conversational rather than just robotic. It's not just about the code, it's about crafting the experience.

Project timeline

It took me approximately one hour to complete this project. The most challenging part was understanding how to properly balance the confidence threshold with the fallback responses to make the bot feel natural and it was most rewarding to see **BankerBot** successfully handle greetings and provide helpful clarification when it didn't understand an input.



Setting up a Lex chatbot

Approach to chatbot setup

In this step, I am setting up a new chatbot from scratch on Amazon Lex because it provides the essential foundation for building my **BankerBot**, allowing me to leverage AWS's pre-built AI capabilities instead of managing complex infrastructure myself.

Chatbot creation process

I created my chatbot from scratch using Amazon Lex. Setting it up only took me a few minutes!

IAM role configuration

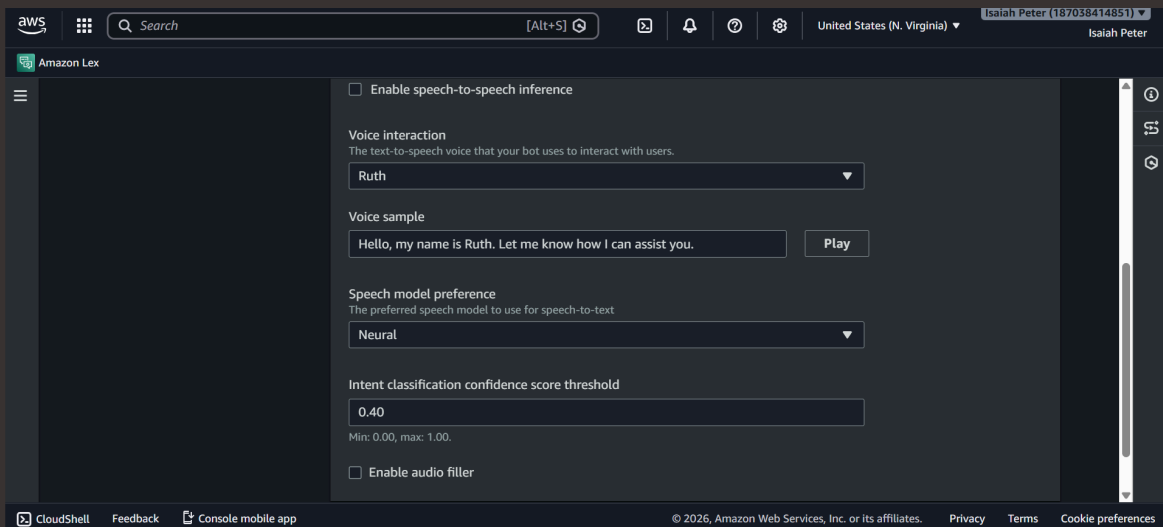
While creating my chatbot, I also created a role with basic permissions because Amazon Lex needs the authority to call other AWS services on my behalf. Specifically, I'll be integrating it with another service called (Lambda) later in this project series to handle the backend logic for my banking functions!

Intent classification confidence score

The intent classification confidence score threshold represents the minimum level of certainty required for the chatbot to act on a user's request. By keeping the default value of **0.40**, I am telling BankerBot that it must be at least 40% confident it understands what the user is asking before it provides a specific response.



If the user's input is too ambiguous or doesn't match my defined intents closely enough resulting in a score below this threshold the bot will trigger the (FallbackIntent) instead of guessing incorrectly.





Intents

In this step, I am training my BankerBot to greet users. Since I'm building this chatbot from scratch to learn exactly how it works, I am defining a specific "intent" a goal for the bot called `WelcomeIntent`. I'm teaching the bot to recognize phrases like "Hi" or "Can you help me?" and configuring it to respond with a friendly introduction. By doing this manually, I'm gaining hands on experience with how intents, utterances, and responses come together to create a conversational flow.

Creating my first intent

An intent is what the user is trying to achieve in their conversation with the chatbot. For example, checking a bank account balance, booking a flight, ordering food.

I created my first intent, `WelcomeIntent`, to set the tone, establish the bot persona and manage customers expectations from the get go.

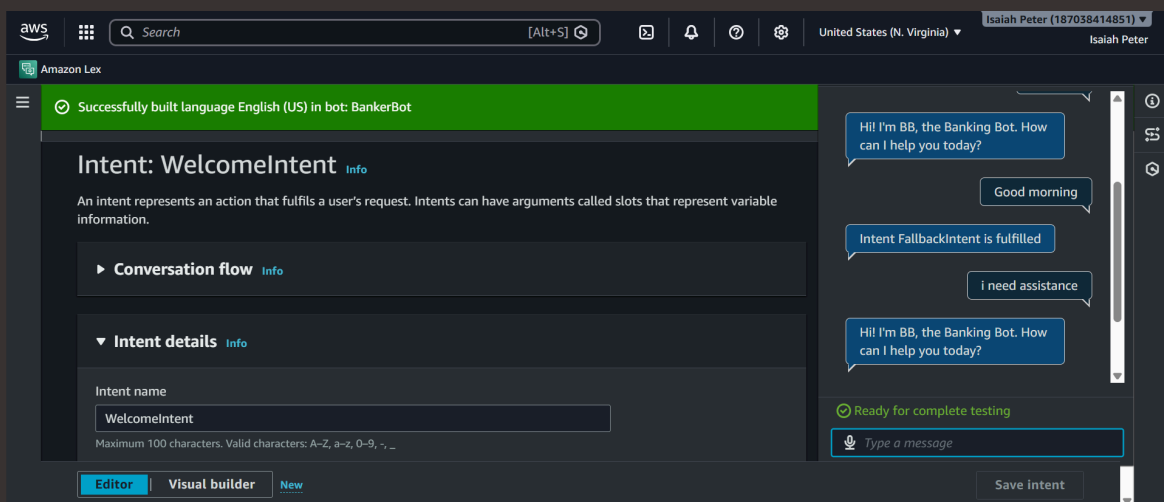
Testing chatbot responses

I launched and tested my chatbot, which could respond successfully with the welcome intent if I enter greetings like hello, i need assistance.



Handling unrecognized inputs

My chatbot returned the error message 'Intent FallbackIntent is fulfilled' when I entered Good Morning. This error message occurred because it does not recognise the intent and could not confidently give a response.





Configuring FallbackIntent

In this step, I am customizing the **FallbackIntent** to make my **BankerBot** more user friendly. Currently, if the bot doesn't understand a request, it returns a confusing default error message. I am updating this to send a clear, helpful response that explains what the bot actually can do like checking account balances, transferring funds, or making payments and adding variations to make the bot sound more natural and conversational when it needs clarification!

When FallbackIntent triggers

FallbackIntent gets triggered when my chatbot's confidence score for all my defined intents falls below the threshold of 0.40. Essentially, if the user says something that the bot doesn't clearly recognize based on the utterances I've trained it with, it defaults to this intent to ask for clarification.

I configured the **FallbackIntent** to ensure my **BankerBot** handles misunderstandings gracefully and remains user-friendly. By customizing this intent, I've replaced the confusing default error message with a clear, helpful response. Now, whenever the bot can't confidently understand a user's request (meaning the confidence score is below 0.40), it will politely explain what it *can* actually do such as checking balances, transferring funds, or making payments and offer a few variations to keep the conversation natural and encouraging rather than letting the user feel stuck.

My FallbackIntent configuration

To configure the **FallbackIntent**, I navigated to the intent in my left hand navigation panel and focused on the **Closing responses** section. I replaced the default, confusing error message with a clear, helpful response: "Sorry I am having trouble understanding."



Can you describe what you'd like to do in a few words? I can help you find your account balance, transfer funds and make a payment."* I also expanded the **Variations** section to add another way for the bot to ask for clarification, which makes the conversation sound more natural and dynamic whenever the bot isn't confident enough to proceed. After making these changes, I saved the intent and built it to ensure the new responses were active in my **BankerBot**.

Variations are alternative ways for **BankerBot** to phrase the same message. When my chatbot needs to return a **FallbackIntent** response, it will randomly select one message from the list I've created. By providing multiple variations instead of just a single fixed response, I make the bot's behavior feel more dynamic, natural, and conversational to the end user, preventing it from sounding like a broken record.



FallbackIntent with Variations

The screenshot displays the Amazon Lex console interface. At the top, a green banner indicates 'Successfully built language English (US) in bot: BankerBot'. The left sidebar shows the 'Amazon Lex' navigation menu with options like 'Back to intents list (2)', a search bar, and a list of intents including 'FallbackIntent' and 'WelcomeIntent'. The main content area is titled 'Intent: FallbackIntent' and includes a description: 'An intent represents an action that fulfills a user's request. Intents can have arguments called slots.' Below this, there are sections for 'Conversation flow' and 'Intent details'. The 'Intent details' section is expanded, showing the 'Editor' tab. On the right side of the console, a chat window simulates a conversation with the bot. The bot's responses are: 'Hmm could you try rephrasing that? I can help you find your account balance, transfer funds and make a payment.' and 'Sorry I am having trouble understanding. Can you describe what you'd like to do in a few words? I can help you find your account balance, transfer funds and make a payment.' The chat window also shows a user input 'Good morning' and a status 'Ready for complete testing' with a 'Save intent' button.



Initial Responses

In this secret mission, I am completing a project extension by demonstrating my ability to spice up fallbackintent, add variations to initial response, document the "why" and "how" behind my work. I am taking a moment to reflect on my progress with **BankerBot**, articulating my goals and the technical steps I've taken so far. By doing this, I am building a strong narrative for my professional portfolio, showing that I don't just build features, but truly understand the purpose and impact of the AI systems I create.

Setting up initial responses

Actually, my chatbot sends its initial greeting for the WelcomeIntent, not the FallbackIntent. I configured the WelcomeIntent specifically to trigger as soon as the conversation starts to provide a friendly introduction. The FallbackIntent, on the other hand, is only triggered after the initial interaction has begun, and only when the bot fails to understand a specific input (i.e., when the confidence score drops below 0.40).

Initial response implementation

I've set up initial responses for **BankerBot** to greet users and handle misunderstandings gracefully: * **WelcomeIntent:** I configured the **Closing response** to: "Hi! I'm BB, the Banking Bot. How can I help you today?" This ensures users get a friendly, clear introduction immediately. * **FallbackIntent:** I updated the **Closing response** to: "Sorry I am having trouble understanding. Can you describe what you'd like to do in a few words? I can help you find your account balance, transfer funds and make a payment." I also added **Variations** to this message to keep interactions natural when the bot's confidence score is below 0.40. I've built the bot to ensure these responses are active.



United States (N. Virginia)

Isaiah Peter (187038414851)

Search

[Alt+S] Ask Amazon Q

Amazon Lex

Successfully built language English (US) in bot: BankerBot

Fulfilment

Info

Run a lambda function to fulfil the intent and inform users of the status when it's complete.

Active

On successful fulfilment

Message: -

In case of failure

Message: -

Closing response

Info

You can define the response when closing the intent.

Active

Response sent to the user after the intent is fulfilled

Editor

Visual builder

New

Save intent

my Account info

Hmmm this is interesting...

Hmm could you try rephrasing that? I can help you find your account balance, transfer funds and make a payment.

my bank statement

One moment...

Ready for complete testing

Type a message



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

